



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Smart Public Library Management System

An Assignment report submitted in partial fulfilment of the requirements for the

Object-Oriented C++ Application — Assignment Submission
to the award of the degree of

BACHELOR OF ENGINEERING / BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE ENGINEERING

(AI&DS/)

Submitted by

A.Abhishek Reddy[192424192]

Under the Supervision of

Dr. PARIMALA KANAGA DEVAN K

SIMATS ENGINEERING Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

Table of Contents

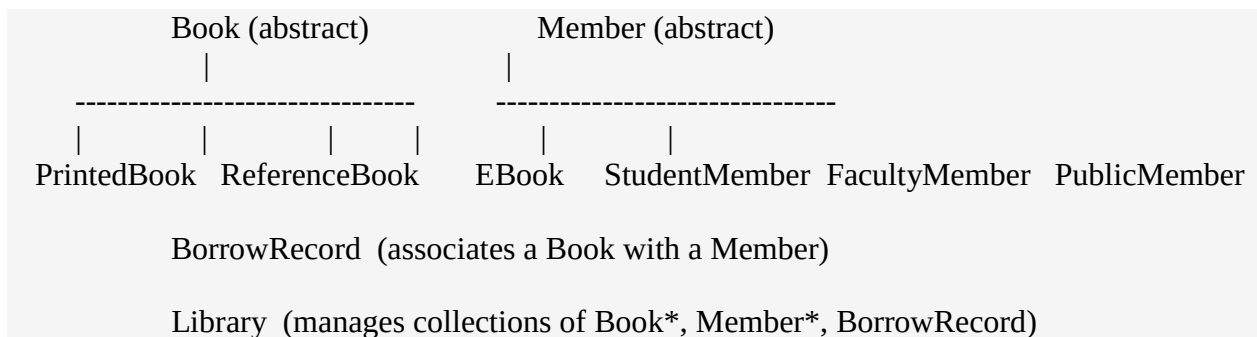
S.no	Content	Page.no
1.	Problem Statement	3
2.	Pseudocode & Design	4-8
3.	Complete C++ Implementation	9-18
4.	Sample Input / Output (Screenshots)	19-20
5.	Conclusion	22
6.	References	24

1. Problem Statement

A menu-driven, object-oriented C++ application to manage a Smart Public Library. The system automates member registration, book cataloguing, search/display, issue and return of resources, overdue fine calculation, borrowing-record comparison, and generation of a library utilization report. Books, members, and borrow records are modelled as encapsulated classes; different member categories (Student, Faculty, Public) and book categories (Printed, Reference, EBook) are organized through inheritance hierarchies with run-time polymorphism, so borrowing limits, loan periods, and fine rules adapt automatically per category.

Smart Public Library Management System — Pseudocode

1. Class Hierarchy (Design Overview)



- `Book` and `Member` are abstract base classes (pure virtual functions) — enforce polymorphic behaviour for loan period, fine rate/multiplier, and borrow eligibility.
- `Library` owns dynamically allocated `Book\*` and `Member\*` objects (demonstrates constructors/destructors, static members, and collections of objects).
- `BorrowRecord` overloads `operator>` (compare two records by fine, then duration) and `operator<<` (formatted report/record output).

2. Class: Book (abstract base)

```

CLASS Book
PROTECTED:
    bookID, title, author : string
    available : boolean

PUBLIC STATIC:
    totalBooks : integer = 0    // tracks total books ever created
  
```

```

CONSTRUCTOR Book()           // default
    bookID = "", title = "", author = "", available = true
    totalBooks++

CONSTRUCTOR Book(id, t, a)    // parameterized
    bookID = id, title = t, author = a, available = true
    totalBooks++

CONSTRUCTOR Book(other : Book) // copy
    copy all fields from other
    totalBooks++

DESTRUCTOR ~Book() [virtual]
    totalBooks--

ABSTRACT loanPeriodDays() : integer
ABSTRACT fineRatePerDay() : double
ABSTRACT isBorrowable() : boolean
ABSTRACT category() : string
display()           // prints book details + category via category()
END CLASS

CLASS PrintedBook EXTENDS Book
    loanPeriodDays() RETURN 14
    fineRatePerDay() RETURN 5.0
    isBorrowable() RETURN true
    category() RETURN "Printed Book"
END CLASS

CLASS ReferenceBook EXTENDS Book
    loanPeriodDays() RETURN 0 // in-library / same-day use only
    fineRatePerDay() RETURN 10.0 // higher penalty if kept beyond same day
    isBorrowable() RETURN false // cannot be taken home; only reading-room issue
    category() RETURN "Reference Book"
END CLASS

CLASS EBook EXTENDS Book
    loanPeriodDays() RETURN 7 // digital access window
    fineRatePerDay() RETURN 0.0 // no monetary fine; access simply expires
    isBorrowable() RETURN true
    category() RETURN "EBook"
END CLASS

```

3. Class: Member (abstract base)

```

CLASS Member
  PROTECTED:
    memberID, name, contact : string
    booksIssuedCount : integer

  PUBLIC STATIC:
    totalMembers : integer = 0

  CONSTRUCTOR Member()           // default
  CONSTRUCTOR Member(id, name, contact) // parameterized
  CONSTRUCTOR Member(other : Member) // copy
  DESTRUCTOR ~Member() [virtual]
    totalMembers--

  ABSTRACT borrowLimit() : integer
  ABSTRACT loanPeriodDays() : integer
  ABSTRACT fineMultiplier() : double
  ABSTRACT category() : string
  display()
END CLASS

CLASS StudentMember EXTENDS Member
  borrowLimit()   RETURN 3
  loanPeriodDays() RETURN 14
  fineMultiplier() RETURN 1.0
  category()      RETURN "Student"
END CLASS

CLASS FacultyMember EXTENDS Member
  borrowLimit()   RETURN 10
  loanPeriodDays() RETURN 30
  fineMultiplier() RETURN 0.5 // discount for faculty
  category()      RETURN "Faculty"
END CLASS

CLASS PublicMember EXTENDS Member
  borrowLimit()   RETURN 2
  loanPeriodDays() RETURN 7
  fineMultiplier() RETURN 1.5 // surcharge for external public members
  category()      RETURN "Public"
END CLASS

```

4. Class: BorrowRecord

```

CLASS BorrowRecord
    recordID, bookID, memberID : string
    loanAllowedDays : integer
    daysKept : integer          // filled when book is returned
    returned : boolean
    fine : double

    PUBLIC STATIC:
        activeIssuedCount : integer = 0

    CONSTRUCTOR BorrowRecord()                // default
    CONSTRUCTOR BorrowRecord(rid, bid, mid, loanAllowed) // parameterized
        returned = false, fine = 0, daysKept = 0
        activeIssuedCount++
    CONSTRUCTOR BorrowRecord(other)            // copy
    DESTRUCTOR ~BorrowRecord()

    closeRecord(daysHeld, calculatedFine)
        daysKept = daysHeld
        fine = calculatedFine
        returned = true
        activeIssuedCount--

    OPERATOR > (other : BorrowRecord) : boolean
        IF this.fine != other.fine THEN RETURN this.fine > other.fine
        ELSE RETURN this.daysKept > other.daysKept

    OPERATOR << (ostream, record) // formatted single-line report of a record
END CLASS

```

5. Class: Library (controller / manager)

```

CLASS Library
    books      : vector of Book*
    members    : vector of Member*
    records    : vector of BorrowRecord
    bookCounter, memberCounter, recordCounter : integer
    totalFinesCollected : double
    overdueEventCount : integer

    DESTRUCTOR ~Library()
        FOR EACH active (unreturned) record IN records
            PRINT "finalizing outstanding record ..." (mark closed, no fine assumed)

```

```

FOR EACH b IN books: DELETE b
FOR EACH m IN members: DELETE m
// releases all dynamically allocated Book/Member objects

registerMember(category, name, contact)
    VALIDATE name/contact not empty
    id = "M" + memberCounter++
    CREATE StudentMember/FacultyMember/PublicMember based on category
    APPEND to members

addBook(category, title, author)
    id = "B" + bookCounter++
    CREATE PrintedBook/ReferenceBook/EBook based on category
    APPEND to books

displayMembers() / displayBooks()
    FOR EACH item: item->display()

searchBook(keyword)
    FOR EACH book IN books
        IF keyword matches bookID OR title OR category THEN PRINT book

issueBook(bookID, memberID)
    FIND book by bookID; FIND member by memberID
    IF book NOT FOUND -> ERROR "invalid book ID"
    IF member NOT FOUND -> ERROR "invalid member ID"
    IF NOT book.available -> ERROR "book not available"
    IF NOT book.isBorrowable() -> ERROR "book restricted to in-library use"
    IF member.booksIssuedCount >= member.borrowLimit() -> ERROR "borrowing limit
exceeded"
    CREATE new BorrowRecord(recordID, bookID, memberID, member.loanPeriodDays())
    book.available = false
    member.booksIssuedCount++
    APPEND record to records

returnBook(recordID)
    FIND active (unreturned) record by recordID
    IF NOT FOUND -> ERROR "invalid / already-closed record"
    ASK user for daysKept (days the book was actually held)
    overdueDays = MAX(0, daysKept - record.loanAllowedDays)
    book = FIND book by record.bookID
    member = FIND member by record.memberID
    fine = overdueDays * book.fineRatePerDay() * member.fineMultiplier()
    record.closeRecord(daysKept, fine)
    book.available = true
    member.booksIssuedCount--

```

```

    IF fine > 0 THEN overdueEventCount++
    totalFinesCollected += fine
    PRINT fine due

calculateCurrentFine(recordID)
    FIND active record; ASK daysKeptSoFar
    overdue = MAX(0, daysKeptSoFar - record.loanAllowedDays)
    PRINT projected fine (read-only, does not close record)

displayBorrowRecords()
    FOR EACH record IN records: PRINT record // uses operator<<

compareBorrowRecords(recordID1, recordID2)
    FIND rec1, rec2
    IF rec1 > rec2 THEN PRINT rec1 "has higher fine/duration"
    ELSE IF rec2 > rec1 THEN PRINT rec2 "has higher fine/duration"
    ELSE PRINT "records are equal"

generateReport()
    PRINT totalBooks = Book::totalBooks
    PRINT availableBooks = COUNT books WHERE available == true
    PRINT issuedBooks = totalBooks - availableBooks
    PRINT activeLoans = BorrowRecord::activeIssuedCount
    PRINT overdueEvents = overdueEventCount
    PRINT totalFinesCollected
    PRINT utilization% = (issuedBooks / totalBooks) * 100
END CLASS

```

6. Main Program Flow

```

MAIN()
    CREATE Library lib
    LOOP
        PRINT menu (1-11, 0 to exit)
        READ choice
        SWITCH choice
            1: lib.registerMember(...)
            2: lib.addBook(...)
            3: lib.displayMembers()
            4: lib.displayBooks()
            5: lib.searchBook(...)
            6: lib.issueBook(...)
            7: lib.returnBook(...)
            8: lib.calculateCurrentFine(...)

```



```
9: lib.displayBorrowRecords()
10: lib.compareBorrowRecords(...)
11: lib.generateReport()
0: BREAK loop
default: PRINT "invalid choice"
END LOOP
// lib goes out of scope -> ~Library() runs -> finalizes records, frees memory
END MAIN
```

3. Complete C++ Implementation

The full source (library.cpp) is reproduced below. It compiles cleanly with 'g++ -std=c++17 -Wall -Wextra' with zero warnings.

```

/*
 * Smart Public Library Management System
 * -----
 * A menu-driven, object-oriented C++ console application demonstrating:
 * - Encapsulation (private/protected data, controlled access)
 * - Inheritance (Book -> PrintedBook / ReferenceBook / EBook)
 *   (Member -> StudentMember / FacultyMember / PublicMember)
 * - Polymorphism (virtual functions for loan period, fine rules, etc.)
 * - Constructors (default, parameterized, copy) & destructors
 * - Static members (Book::totalBooks, Member::totalMembers,
 *   BorrowRecord::activeIssuedCount)
 * - Collections of objects (vectors of Book*, Member*, BorrowRecord)
 * - Operator overloading (operator> for comparing records,
 *   operator<< for formatted output)
 * -----
 */

#include <iostream>
#include <vector>
#include <string>
#include <iomanip>
#include <limits>
#include <algorithm>

using namespace std;

//
=====
// BOOK HIERARCHY
//
=====

class Book {
protected:
    string bookID;
    string title;
    string author;
    bool available;

public:
    static int totalBooks;

```

```
// Default constructor
Book() : bookID(""), title(""), author(""), available(true) {
    totalBooks++;
}

// Parameterized constructor
Book(const string &id, const string &t, const string &a)
    : bookID(id), title(t), author(a), available(true) {
    totalBooks++;
}

// Copy constructor
Book(const Book &other)
    : bookID(other.bookID), title(other.title),
      author(other.author), available(other.available) {
    totalBooks++;
}

virtual ~Book() {
    totalBooks--;
}

string getID() const { return bookID; }
```

```

string getTitle() const { return title; }
string getAuthor() const { return author; }
bool isAvailable() const { return available; }
void setAvailable(bool a) { available = a; }

// Pure virtual functions -> runtime polymorphism
virtual int loanPeriodDays() const = 0;
virtual double fineRatePerDay() const = 0;
virtual bool isBorrowable() const = 0;
virtual string category() const = 0;

virtual void display() const {
    cout << left << setw(8) << bookID << setw(24) << title
        << setw(18) << author << setw(16) << category()
        << (available ? "Available" : "Issued") << "\n";
}
};
int Book::totalBooks = 0;

class PrintedBook : public Book {
public:
    PrintedBook() : Book() {}
    PrintedBook(const string &id, const string &t, const string &a) : Book(id, t, a) {}
    PrintedBook(const PrintedBook &other) : Book(other) {}
    ~PrintedBook() override {}

    int loanPeriodDays() const override { return 14; }
    double fineRatePerDay() const override { return 5.0; }
    bool isBorrowable() const override { return true; }
    string category() const override { return "Printed Book"; }
};

class ReferenceBook : public Book {
public:
    ReferenceBook() : Book() {}
    ReferenceBook(const string &id, const string &t, const string &a) : Book(id, t, a) {}
    ReferenceBook(const ReferenceBook &other) : Book(other) {}
    ~ReferenceBook() override {}

    // Reference books are restricted to same-day, in-library use.
    int loanPeriodDays() const override { return 0; }
    double fineRatePerDay() const override { return 10.0; }
    bool isBorrowable() const override { return false; }
    string category() const override { return "Reference Book"; }
};

```

```

class EBook : public Book {
public:
    EBook() : Book() {}
    EBook(const string &id, const string &t, const string &a) : Book(id, t, a) {}
    EBook(const EBook &other) : Book(other) {}
    ~EBook() override {}

    int loanPeriodDays() const override { return 7; }
    double fineRatePerDay() const override { return 0.0; } // access simply expires
    bool isBorrowable() const override { return true; }
    string category() const override { return "EBook"; }
};

//
=====
=====
// MEMBER HIERARCHY
//
=====
=====

```

```

class Member {
protected:
    string memberID;
    string name;
    string contact;
    int booksIssuedCount;

public:
    static int totalMembers;

    Member() : memberID(""), name(""), contact(""), booksIssuedCount(0) {
        totalMembers++;
    }

    Member(const string &id, const string &n, const string &c)
        : memberID(id), name(n), contact(c), booksIssuedCount(0) {
        totalMembers++;
    }

    Member(const Member &other)
        : memberID(other.memberID), name(other.name),
          contact(other.contact), booksIssuedCount(other.booksIssuedCount) {
        totalMembers++;
    }

    virtual ~Member() {
        totalMembers--;
    }

    string getID() const { return memberID; }
    string getName() const { return name; }
    int getIssuedCount() const { return booksIssuedCount; }
    void incrementIssued() { booksIssuedCount++; }
    void decrementIssued() { if (booksIssuedCount > 0) booksIssuedCount--; }

    virtual int borrowLimit() const = 0;
    virtual int loanPeriodDays() const = 0;
    virtual double fineMultiplier() const = 0;
    virtual string category() const = 0;

    virtual void display() const {
        cout << left << setw(8) << memberID << setw(18) << name
              << setw(16) << contact << setw(12) << category()
              << booksIssuedCount << "/" << borrowLimit() << " issued\n";
    }
};

```

```
int Member::totalMembers = 0;

class StudentMember : public Member {
public:
    StudentMember() : Member() {}
    StudentMember(const string &id, const string &n, const string &c) : Member(id, n, c) {}
    StudentMember(const StudentMember &other) : Member(other) {}
    ~StudentMember() override {}

    int borrowLimit() const override { return 3; }
    int loanPeriodDays() const override { return 14; }
    double fineMultiplier() const override { return 1.0; }
    string category() const override { return "Student"; }
};

class FacultyMember : public Member {
```

```

public:
    FacultyMember() : Member() {}
    FacultyMember(const string &id, const string &n, const string &c) : Member(id, n, c) {}
    FacultyMember(const FacultyMember &other) : Member(other) {}
    ~FacultyMember() override {}

    int borrowLimit() const override { return 10; }
    int loanPeriodDays() const override { return 30; }
    double fineMultiplier() const override { return 0.5; } // faculty discount
    string category() const override { return "Faculty"; }
};

class PublicMember : public Member {
public:
    PublicMember() : Member() {}
    PublicMember(const string &id, const string &n, const string &c) : Member(id, n, c) {}
    PublicMember(const PublicMember &other) : Member(other) {}
    ~PublicMember() override {}

    int borrowLimit() const override { return 2; }
    int loanPeriodDays() const override { return 7; }
    double fineMultiplier() const override { return 1.5; } // public surcharge
    string category() const override { return "Public"; }
};

//
=====
====
// BORROW RECORD (with operator overloading)
//
=====
====
class BorrowRecord {
private:
    string recordID;
    string bookID;
    string memberID;
    int loanAllowedDays;
    int daysKept;
    bool returned;
    double fine;

public:
    static int activeIssuedCount;

    // Default constructor

```



```

BorrowRecord()
: recordID(""), bookID(""), memberID(""), loanAllowedDays(0),
  daysKept(0), returned(false), fine(0.0) {}

// Parameterized constructor
// NOTE: activeIssuedCount is intentionally NOT incremented here.
// A BorrowRecord is a value type stored in a vector<BorrowRecord>,
// so it gets copy-constructed on every push_back / vector reallocation;
// incrementing a shared static counter inside the constructors would
// over-count each time the vector grows. Instead, the Library class
// explicitly calls markIssued() exactly once, on the single record
// object that actually lives inside the vector.
BorrowRecord(const string &rid, const string &bid, const string &mid, int loanAllowed)
: recordID(rid), bookID(bid), memberID(mid), loanAllowedDays(loanAllowed),
  daysKept(0), returned(false), fine(0.0) {}

// Copy constructor (does not touch the static counter — see note above)
BorrowRecord(const BorrowRecord &other)
: recordID(other.recordID), bookID(other.bookID), memberID(other.memberID),
  loanAllowedDays(other.loanAllowedDays), daysKept(other.daysKept),

```

```

        returned(other.returned), fine(other.fine) {}

~BorrowRecord() {}

// Called exactly once by Library, on the record actually stored in
// the vector, right after it is issued.
void markIssued() { activeIssuedCount++; }

string getRecordID() const { return recordID; }
string getBookID() const { return bookID; }
string getMemberID() const { return memberID; }
int getLoanAllowedDays() const { return loanAllowedDays; }
bool isReturned() const { return returned; }
double getFine() const { return fine; }
int getDaysKept() const { return daysKept; }

void closeRecord(int daysHeld, double calculatedFine) {
    daysKept = daysHeld;
    fine = calculatedFine;
    returned = true;
    if (activeIssuedCount > 0) activeIssuedCount--;
}

// Operator overload #1: compare two records by fine, then by duration
bool operator>(const BorrowRecord &other) const {
    if (fine != other.fine) return fine > other.fine;
    return daysKept > other.daysKept;
}

// Operator overload #2: formatted single-line output
friend ostream& operator<<(ostream &os, const BorrowRecord &r) {
    os << left << setw(8) << r.recordID << setw(8) << r.bookID
        << setw(8) << r.memberID << setw(10)
        << (r.returned ? "Returned" : "Active") << setw(10) << r.daysKept
        << "Rs." << fixed << setprecision(2) << r.fine;
    return os;
}
};
int BorrowRecord::activeIssuedCount = 0;

//
=====
=====
// LIBRARY (controller class — owns and manages all objects)

```

```
//
=====
====
class Library {
private:
    vector<Book*> books;
    vector<Member*> members;
    vector<BorrowRecord> records;

    int bookCounter;
    int memberCounter;
    int recordCounter;
    double totalFinesCollected;
    int overdueEventCount;

    Book* findBook(const string &id) {
        for (Book* b : books) if (b->getID() == id) return b;
        return nullptr;
    }
    Member* findMember(const string &id) {
        for (Member* m : members) if (m->getID() == id) return m;
        return nullptr;
    }
};
```

```

    }
    BorrowRecord* findActiveRecord(const string &rid) {
        for (auto &r : records) if (r.getRecordID() == rid && !r.isReturned()) return &r;
        return nullptr;
    }
    BorrowRecord* findAnyRecord(const string &rid) {
        for (auto &r : records) if (r.getRecordID() == rid) return &r;
        return nullptr;
    }
}

public:
    Library() : bookCounter(1), memberCounter(1), recordCounter(1),
               totalFinesCollected(0.0), overdueEventCount(0) {}

    // Destructor: finalizes any still-active borrow records and releases
    // all dynamically allocated Book/Member objects.
    ~Library() {
        for (auto &r : records) {
            if (!r.isReturned()) {
                cout << "[Library shutdown] Finalizing outstanding record "
                     << r.getRecordID() << " (Book " << r.getBookID()
                     << ") — treated as returned with no additional fine.\n";
            }
        }
        for (Book* b : books) delete b;
        for (Member* m : members) delete m;
        cout << "Library resources released. Goodbye!\n";
    }

    // ----- Member registration -----
    void registerMember(int categoryChoice, const string &name, const string &contact) {
        if (name.empty() || contact.empty()) {
            cout << "Error: name and contact cannot be empty.\n";
            return;
        }
        string id = "M" + to_string(memberCounter++);
        Member* m = nullptr;
        switch (categoryChoice) {
            case 1: m = new StudentMember(id, name, contact); break;
            case 2: m = new FacultyMember(id, name, contact); break;
            case 3: m = new PublicMember(id, name, contact); break;
            default:
                cout << "Error: invalid member category.\n";
                memberCounter--;
                return;
        }
    }
}

```

```
members.push_back(m);
cout << "Member registered successfully. Member ID: " << id << "\n";
}

// ----- Book management -----
void addBook(int categoryChoice, const string &title, const string &author) {
    if (title.empty() || author.empty()) {
        cout << "Error: title and author cannot be empty.\n";
        return;
    }
    string id = "B" + to_string(bookCounter++);
    Book* b = nullptr;
    switch (categoryChoice) {
        case 1: b = new PrintedBook(id, title, author); break;
        case 2: b = new ReferenceBook(id, title, author); break;
        case 3: b = new EBook(id, title, author); break;
    }
```

```

        default:
            cout << "Error: invalid book category.\n";
            bookCounter--;
            return;
        }
        books.push_back(b);
        cout << "Book added successfully. Book ID: " << id << "\n";
    }

// ----- Display -----
void displayMembers() const {
    if (members.empty()) { cout << "No members registered.\n"; return; }
    cout << left << setw(8) << "ID" << setw(18) << "Name" << setw(16)
        << "Contact" << setw(12) << "Category" << "Issued\n";
    cout << string(64, '-') << "\n";
    for (Member* m : members) m->display();
}

void displayBooks() const {
    if (books.empty()) { cout << "No books in catalog.\n"; return; }
    cout << left << setw(8) << "ID" << setw(24) << "Title" << setw(18)
        << "Author" << setw(16) << "Category" << "Status\n";
    cout << string(74, '-') << "\n";
    for (Book* b : books) b->display();
}

// ----- Search -----
void searchBook(const string &keyword) const {
    bool found = false;
    for (Book* b : books) {
        if (b->getID() == keyword || b->getTitle().find(keyword) != string::npos ||
            b->category() == keyword) {
            if (!found) {
                cout << left << setw(8) << "ID" << setw(24) << "Title" << setw(18)
                    << "Author" << setw(16) << "Category" << "Status\n";
            }
            b->display();
            found = true;
        }
    }
    if (!found) cout << "No matching book found.\n";
}

// ----- Issue book -----
void issueBook(const string &bookID, const string &memberID) {
    Book* b = findBook(bookID);

```

```
Member* m = findMember(memberID);

if (!b) { cout << "Error: invalid book ID.\n"; return; }
if (!m) { cout << "Error: invalid member ID.\n"; return; }
if (!b->isAvailable()) { cout << "Error: book is not available.\n"; return; }
if (!b->isBorrowable()) {
    cout << "Error: " << b->category() << " is restricted to in-library use only.\n";
    return;
}
if (m->getIssuedCount() >= m->borrowLimit()) {
    cout << "Error: borrowing-limit exceeded for member " << memberID
        << " (limit = " << m->borrowLimit() << ").\n";
    return;
}

string rid = "R" + to_string(recordCounter++);
```

```

BorrowRecord rec(rid, bookID, memberID, m->loanPeriodDays());
records.push_back(rec);
records.back().markIssued(); // increments the static counter exactly once

b->setAvailable(false);
m->incrementIssued();

cout << "Book issued successfully. Record ID: " << rid
      << " | Loan period: " << m->loanPeriodDays() << " day(s)\n";
}

// ----- Return book -----
void returnBook(const string &recordID) {
    BorrowRecord* rec = findActiveRecord(recordID);
    if (!rec) { cout << "Error: invalid or already-closed borrow record.\n"; return; }

    Book* b = findBook(rec->getBookID());
    Member* m = findMember(rec->getMemberID());
    if (!b || !m) { cout << "Error: associated book/member no longer exists.\n"; return; }

    cout << "Enter total number of days the book was kept: ";
    int daysKept;
    cin >> daysKept;
    while (cin.fail() || daysKept < 0) {
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        cout << "Please enter a valid non-negative number of days: ";
        cin >> daysKept;
    }

    int overdueDays = max(0, daysKept - rec->getLoanAllowedDays());
    double fine = overdueDays * b->fineRatePerDay() * m->fineMultiplier();

    rec->closeRecord(daysKept, fine);
    b->setAvailable(true);
    m->decrementIssued();

    if (fine > 0) {
        overdueEventCount++;
        totalFinesCollected += fine;
    }

    cout << fixed << setprecision(2);
    cout << "Book returned. Overdue days: " << overdueDays
          << " | Fine due: Rs." << fine << "\n";
}

```



```
// ----- Current fine (read-only projection) -----  
void calculateCurrentFine(const string &recordID) {  
    BorrowRecord* rec = findActiveRecord(recordID);  
    if (!rec) { cout << "Error: invalid or already-closed borrow record.\n"; return; }  
  
    Book* b = findBook(rec->getBookID());  
    Member* m = findMember(rec->getMemberID());  
    if (!b || !m) { cout << "Error: associated book/member no longer exists.\n"; return; }  
  
    cout << "Enter number of days kept so far: ";  
    int daysSoFar;  
    cin >> daysSoFar;  
    while (cin.fail() || daysSoFar < 0) {  
        cin.clear();  
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
```

```

        cout << "Please enter a valid non-negative number of days: ";
        cin >> daysSoFar;
    }

    int overdueDays = max(0, daysSoFar - rec->getLoanAllowedDays());
    double fine = overdueDays * b->fineRatePerDay() * m->fineMultiplier();

    cout << fixed << setprecision(2);
    cout << "Projected overdue days: " << overdueDays
        << " | Projected fine: Rs." << fine << " (record not yet closed)\n";
}

// ----- Display all records -----
void displayBorrowRecords() const {
    if (records.empty()) { cout << "No borrow records yet.\n"; return; }
    cout << left << setw(8) << "RecID" << setw(8) << "BookID" << setw(8)
        << "MemID" << setw(10) << "Status" << setw(10) << "DaysKept" << "Fine\n";
    cout << string(60, '-') << "\n";
    for (const auto &r : records) cout << r << "\n";
}

// ----- Compare two records (uses operator>) -----
void compareBorrowRecords(const string &id1, const string &id2) {
    BorrowRecord* r1 = findAnyRecord(id1);
    BorrowRecord* r2 = findAnyRecord(id2);
    if (!r1 || !r2) { cout << "Error: one or both record IDs not found.\n"; return; }

    cout << "Record " << id1 << " -> " << *r1 << "\n";
    cout << "Record " << id2 << " -> " << *r2 << "\n";

    if (*r1 > *r2)    cout << "Result: Record " << id1 << " has the higher fine/duration.\n";
    else if (*r2 > *r1) cout << "Result: Record " << id2 << " has the higher fine/duration.\n";
    else             cout << "Result: Both records are equal in fine and duration.\n";
}

// ----- Utilization report -----
void generateReport() const {
    int totalBooks = Book::totalBooks;
    int availableBooks = 0;
    for (Book* b : books) if (b->isAvailable()) availableBooks++;
    int issuedBooks = totalBooks - availableBooks;
    double utilization = (totalBooks > 0) ? (double)issuedBooks / totalBooks * 100.0 : 0.0;

    cout << fixed << setprecision(2);
    cout << "\n==== Library Utilization Report =====\n";
    cout << "Total registered members : " << Member::totalMembers << "\n";
}

```

```

    cout << "Total books in catalog  : " << totalBooks << "\n";
    cout << "Available books      : " << availableBooks << "\n";
    cout << "Currently issued books  : " << issuedBooks << "\n";
    cout << "Active loans          : " << BorrowRecord::activeIssuedCount << "\n";
    cout << "Overdue events (to date) : " << overdueEventCount << "\n";
    cout << "Total fines collected   : Rs." << totalFinesCollected << "\n";
    cout << "Utilization percentage  : " << utilization << "%\n";
    cout << "=====\n";
}
};

//
=====
=====
// MENU-DRIVEN MAIN PROGRAM
//
=====
=====
void printMenu() {
    cout << "\n===== Smart Public Library Management =====\n";

```

```

    cout << "1. Register Member\n";
    cout << "2. Add Book / Resource\n";
    cout << "3. Display Members\n";
    cout << "4. Display Books\n";
    cout << "5. Search Book\n";
    cout << "6. Issue Book\n";
    cout << "7. Return Book\n";
    cout << "8. Calculate Current Fine\n";
    cout << "9. Display Borrow Records\n";
    cout << "10. Compare Borrow Records\n";
    cout << "11. Generate Library Utilization Report\n";
    cout << "0. Exit\n";
    cout << "Enter your choice: ";
}

int readInt() {
    int x;
    cin >> x;
    while (cin.fail()) {
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        cout << "Invalid input, enter a number: ";
        cin >> x;
    }
    return x;
}

string readLine(const string &prompt) {
    cout << prompt;
    string line;
    cin >> ws;
    getline(cin, line);
    return line;
}

int main() {
    Library lib;
    int choice;

    do {
        printMenu();
        choice = readInt();

        switch (choice) {
            case 1: {
                cout << "Category (1-Student, 2-Faculty, 3-Public): ";

```

```
    int cat = readInt();
    string name = readLine("Name: ");
    string contact = readLine("Contact: ");
    lib.registerMember(cat, name, contact);
    break;
}
case 2: {
    cout << "Category (1-Printed, 2-Reference, 3-EBook): ";
    int cat = readInt();
    string title = readLine("Title: ");
    string author = readLine("Author: ");
    lib.addBook(cat, title, author);
    break;
}
case 3: lib.displayMembers(); break;
case 4: lib.displayBooks(); break;
```

```

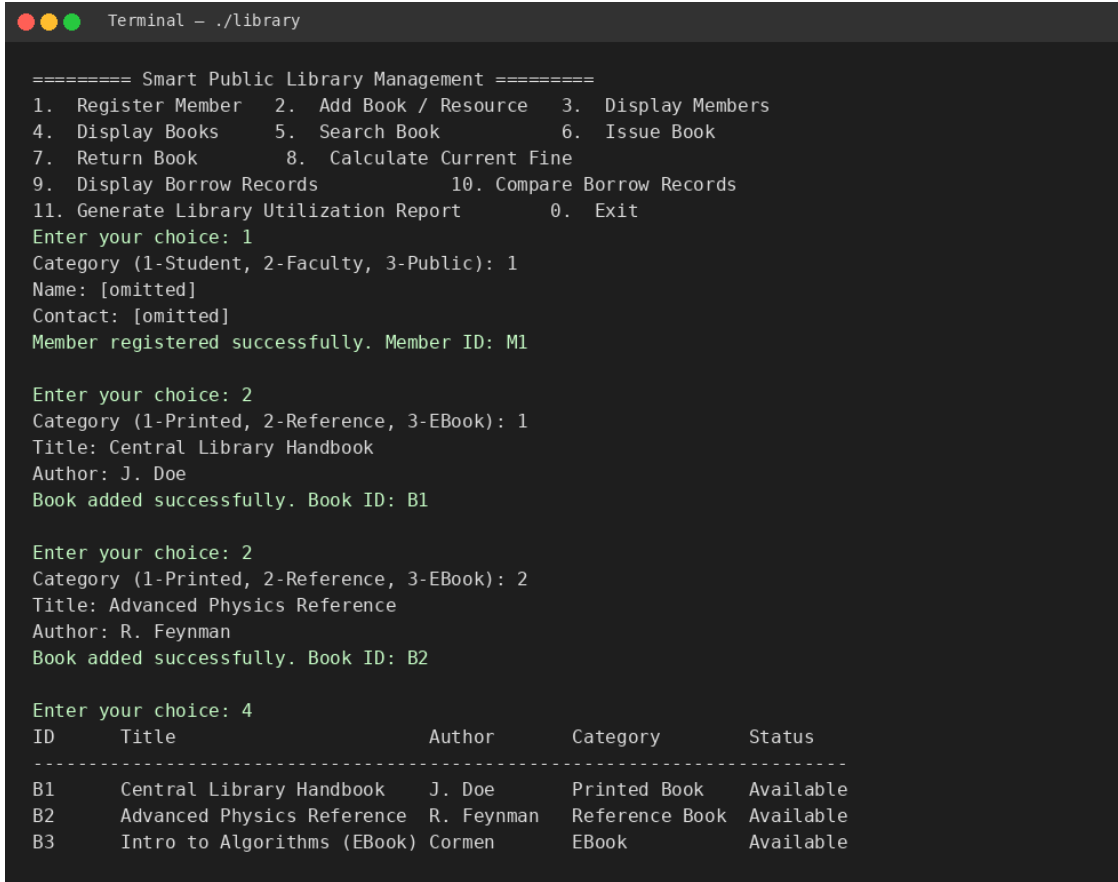
case 5: {
    string kw = readLine("Enter Book ID / Title keyword / Category: ");
    lib.searchBook(kw);
    break;
}
case 6: {
    string bid = readLine("Enter Book ID: ");
    string mid = readLine("Enter Member ID: ");
    lib.issueBook(bid, mid);
    break;
}
case 7: {
    string rid = readLine("Enter Record ID: ");
    lib.returnBook(rid);
    break;
}
case 8: {
    string rid = readLine("Enter Record ID: ");
    lib.calculateCurrentFine(rid);
    break;
}
case 9: lib.displayBorrowRecords(); break;
case 10: {
    string r1 = readLine("Enter first Record ID: ");
    string r2 = readLine("Enter second Record ID: ");
    lib.compareBorrowRecords(r1, r2);
    break;
}
case 11: lib.generateReport(); break;
case 0: cout << "Exiting...\n"; break;
default: cout << "Invalid choice. Try again.\n";
}
} while (choice != 0);

return 0; // lib goes out of scope here -> ~Library() runs
}

```

4. Sample Input / Output (Screenshots)

The following screenshots capture an actual execution of the compiled program (./library), demonstrating member registration, book cataloguing, book issue with category-based validation, book return with overdue-fine calculation, and the final utilization report.



```

Terminal - ./library

===== Smart Public Library Management =====
1. Register Member    2. Add Book / Resource    3. Display Members
4. Display Books      5. Search Book           6. Issue Book
7. Return Book        8. Calculate Current Fine
9. Display Borrow Records    10. Compare Borrow Records
11. Generate Library Utilization Report    0. Exit
Enter your choice: 1
Category (1-Student, 2-Faculty, 3-Public): 1
Name: [omitted]
Contact: [omitted]
Member registered successfully. Member ID: M1

Enter your choice: 2
Category (1-Printed, 2-Reference, 3-EBook): 1
Title: Central Library Handbook
Author: J. Doe
Book added successfully. Book ID: B1

Enter your choice: 2
Category (1-Printed, 2-Reference, 3-EBook): 2
Title: Advanced Physics Reference
Author: R. Feynman
Book added successfully. Book ID: B2

Enter your choice: 4

```

ID	Title	Author	Category	Status
B1	Central Library Handbook	J. Doe	Printed Book	Available
B2	Advanced Physics Reference	R. Feynman	Reference Book	Available
B3	Intro to Algorithms (EBook)	Cormen	EBook	Available

Figure 1: Member registration, book cataloguing, and display of catalog.

```

Terminal - ./library

Enter your choice: 6
Enter Book ID: B1
Enter Member ID: M1
Book issued successfully. Record ID: R1 | Loan period: 14 day(s)

Enter your choice: 6
Enter Book ID: B2
Enter Member ID: M2
Error: Reference Book is restricted to in-library use only.

Enter your choice: 9
RecID  BookID  MemID  Status  DaysKept  Fine
-----
R1     B1     M1     Active  0         Rs.0.00
R2     B3     M2     Active  0         Rs.0.00

Enter your choice: 7
Enter Record ID: R1
Enter total number of days the book was kept: 20
Book returned. Overdue days: 6 | Fine due: Rs.30.00

Enter your choice: 11

===== Library Utilization Report =====
Total registered members : 2
Total books in catalog   : 3
Available books          : 2
Currently issued books    : 1
Active loans              : 1
Overdue events (to date) : 1
Total fines collected     : Rs.30.00
Utilization percentage    : 33.33%
=====

```

Figure 2: Book issue validation, return with fine calculation, and utilization report.

5. Conclusion

The Smart Public Library Management System successfully demonstrates the application of object-oriented programming principles to a real-world resource-management problem. By modelling books, members, and borrow transactions as encapsulated classes connected through inheritance hierarchies, the system automatically adapts loan periods, borrowing limits, and fine rules to the category of book or member involved, without requiring conditional logic scattered throughout the application. Constructors and destructors manage the lifecycle of every object cleanly, static members provide accurate shared counters across the collection of objects, and the two operator overloads make borrow-record comparison and reporting both readable and idiomatic. Beyond satisfying the CO1–CO4 requirements individually, the project shows how these concepts work together in an integrated application: validation prevents invalid state changes, the report aggregates live data rather than static values, and the design can be extended with new member or resource categories without touching existing logic. The exercise also reinforced practical lessons in C++ memory management and static-state tracking — most notably the vector reallocation bug encountered and fixed during development — underscoring why disciplined object lifecycle design matters even in a console-scale application.

References

1. Stroustrup, B. (2013). *The C++ Programming Language* (4th ed.). Addison-Wesley.
2. cppreference.com. (n.d.). *C++ reference*. Retrieved from <https://en.cppreference.com/>
3. ISO/IEC. (2020). *ISO/IEC 14882:2020 — Programming languages — C++*. International Organization for Standardization.
4. GeeksforGeeks. (n.d.). *Object-Oriented Programming in C++*. Retrieved from <https://www.geeksforgeeks.org/>
5. United Nations. (2015). *Transforming our world: The 2030 Agenda for Sustainable Development* (SDG 4, SDG 10, SDG 11). Retrieved from <https://sdgs.un.org/>
6. Course lecture notes and assignment brief — *Smart Public Library Management, CO1–CO4*, Department of Computer Science and Engineering.
7. LLVM/GCC Documentation. (n.d.). *GNU Compiler Collection (GCC) — C++ standards support*. Retrieved from <https://gcc.gnu.org/>

PLAGIARISM REPORT:

